

Python: Standard Library Cheat Sheet

Fundamentals of the standard library — the "batteries included" modules that ship with Python itself, no installs needed. Each section shows the everyday core of one module or task; the full APIs live at docs.python.org. Language topics stay on the basics and advanced sheets.

File & Console I/O

The basics sheet introduces `open()`; this is the fuller picture — appending, binary mode, and the standard streams.

```
import sys

# Console: print() writes to STDOUT; send errors to STDERR
print("normal output")          # -> stdout
print("something went wrong", file=sys.stderr)  # -> stderr
sys.stdout.write("raw write\n")  # print() without separators/newline logic
# input() reads a line from stdin (see the getting-started sheet)

# The standard streams ARE ordinary file objects — the same type open()
# returns, pre-opened by the interpreter. The whole file interface works,
# and they fit anywhere a file object is expected (e.g. print(file=...)).
type(sys.stdout)                # <class '_io.TextIOWrapper'> (when a terminal)
sys.stderr.write("oops\n")       # 5 -> write() returns the chars written
sys.stdout.writable()            # True   (stdin: read-only; stdout/err: write-only)
sys.stdin.readable()            # True
# text = sys.stdin.read()        # read stdin until EOF — would sit waiting here
# for line in sys.stdin:         # ...or iterate it line by line, like any file
# Never close() them — the interpreter owns their lifetime.

# Text files: pass an explicit encoding, and let `with` close the file
with open("notes.txt", "w", encoding="utf-8") as f:  # w = write (TRUNCATES)
    f.write("first line\n")

with open("notes.txt", "a", encoding="utf-8") as f:  # a = append to the end
    f.write("second line\n")

with open("notes.txt", encoding="utf-8") as f:      # mode "r" is the default
    content = f.read()
content      # 'first line\nsecond line\n'

# Read line by line (memory-friendly: never loads the whole file)
with open("notes.txt", encoding="utf-8") as f:
    for line in f:
        print(line.rstrip())    # first line / second line

# Binary files: add "b" to the mode -> bytes in, bytes out, no encoding
with open("blob.bin", "wb") as f:
    f.write(b"\x00\x01\xff")

with open("blob.bin", "rb") as f:
    data = f.read()
data      # b'\x00\x01\xff'
len(data) # 3

# Modes cheat: r read | w write (truncate!) | a append | x create, fail
# if it exists | +b modifiers combine: "rb", "a+", "xb"...
```

Command-Line Arguments

```
import sys

# sys.argv holds the arguments as a list of STRINGS; [0] is the script.
# Running:  python myscript.py input.txt --fast
# gives:    sys.argv == ['myscript.py', 'input.txt', '--fast']

# argparse: declare the arguments, get parsing, validation, and --help
import argparse

parser = argparse.ArgumentParser(description="Process a file")
parser.add_argument("path")           # required positional
parser.add_argument("--fast", action="store_true") # boolean flag
parser.add_argument("--level", type=int, default=1) # typed, with default

# In a real script: args = parser.parse_args() (reads sys.argv)
# An explicit list also works - used here to keep the example runnable:
args = parser.parse_args(["input.txt", "--fast", "--level", "3"])
args.path           # 'input.txt'
args.fast           # True
args.level          # 3 -> already an int, thanks to type=int

# Bad input never reaches your code: argparse prints usage and exits
# parser.parse_args([])      # error: the following arguments are
#                             # required: path -> SystemExit(2)
# parser.parse_args(["-h"])  # prints full help and exits
```